

HW_01

Henry Traynor

Due: 25 August

Assignment

All code can be found here: https://github.com/HenryTraynor/WILD8390/blob/main/Homework/HW_01.Rmd

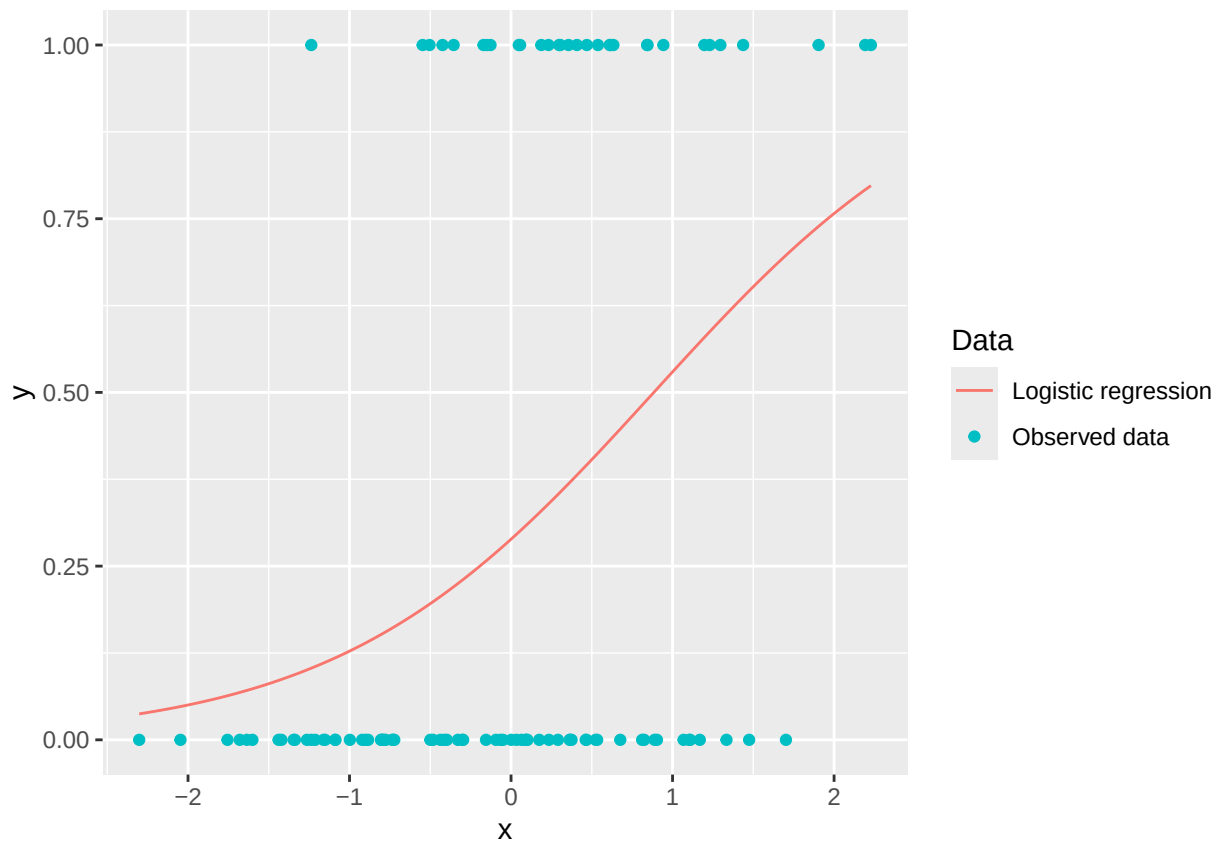
Create a self-contained R script (or better yet an Rmarkdown file) to do the following:

Simulate logistic regression data according to $y_i \sim \text{Bern}(p_i)$ and $\text{logit}(p_i) = \beta_0 + \beta_1 x_i$ with $\beta_0 = -1$ and $\beta_1 = 1$. Generate the covariate using `x <- rnorm(n=100)`.

```
# Set seed so results are the same in development and  
# written answers  
set.seed(314159)  
  
# initialize parameters  
beta_0 <- -1  
beta_1 <- 1  
# generate predictor variable  
x <- rnorm(n = 100)  
# generate response 'plogis()' gives inverse 'logit()' to  
# yield probability  
y <- rbinom(100, size = 1, prob = plogis(beta_0 + beta_1 * x))
```

Fit a logistic regression model to the simulated data using the `glm` function. Create a figure showing x and y with the fitted regression line. Use `predict` to get the fitted line.

```
library(ggplot2)  
  
logistic_regression <- glm(y ~ x, family = binomial)  
# df to store regression data  
predicted_data <- data.frame(x = seq(min(x), max(x), length.out = 100))  
predicted_data$y_hat <- predict(logistic_regression, newdata = predicted_data,  
                               type = "response")  
  
# df to store model input data for plotting  
data <- data.frame(x, y)  
  
ggplot() + geom_point(data = data, aes(x = x, y = y, color = "Observed data")) +  
  geom_line(data = predicted_data, aes(x = x, y = y_hat, color = "Logistic regression")) +  
  labs(x = "x", y = "y", color = "Data")
```



Fit the logistic regression model in JAGS. Use $\text{Normal}(0, \text{var}=10)$ priors for the regression coefficients.

```
library(rjags)
```

```
## Loading required package: coda
```

```
## Linked to JAGS 4.3.1
```

```
## Loaded modules: basemod,bugs
```

```
# Model created using reference of the JAGS Version 4.3.0
# user manual and rjags documentation on rdocumentation.com
# . I will reference what section my code is coming from in
# comments. link:
# https://pilotfiber.dl.sourceforge.net/project/mcmc-jags/Manuals/4.x/jags_user_manual.pdf?viasf=1&fid=
jags_data <- list(y = y, x = x, n = length(y))
```

```
# Chapter 2: model creation Note that dnorm() in JAGS/BUGS
# takes precision, not variance, so I have used
# dnorm(0,1/10) for my priors.
```

```
string <- "model {
  for(i in 1:n) {
    logit(p[i]) <- beta_0 + beta_1 * x[i]
    y[i] ~ dbern(p[i])
  }
}
```

```

beta_0 ~ dnorm(0, 1/10)
beta_1 ~ dnorm(0, 1/10)
}
"

# https://www.rdocumentation.org/packages/rjags/versions/4-17/topics/jags.model
# I was unsure what satisfactory 'n.chains' or 'n.adapt'
# would be so I used values similar to those shown in
# documentation
jags_model <- jags.model(textConnection(string), data = jags_data,
  n.chains = 3, n.adapt = 1000)

```

```

## Compiling model graph
##   Resolving undeclared variables
##   Allocating nodes
## Graph information:
##   Observed stochastic nodes: 100
##   Unobserved stochastic nodes: 2
##   Total graph size: 507
##
## Initializing model

```

```

# burn-in period as described in 4.1 of JAGS manual
update(jags_model, n.iter = 1000)

# posterior samples: 4.1 of JAGS manual
samples <- coda.samples(model = jags_model, variable.names = c("beta_0",
  "beta_1"), n.iter = 5000)

summary(samples)

```

```

##
## Iterations = 2001:7000
## Thinning interval = 1
## Number of chains = 3
## Sample size per chain = 5000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##           Mean      SD Naive SE Time-series SE
## beta_0 -0.9179 0.2462 0.002011      0.002699
## beta_1  1.0496 0.2900 0.002368      0.003155
##
## 2. Quantiles for each variable:
##
##           2.5%      25%      50%      75%      97.5%
## beta_0 -1.4183 -1.0783 -0.9113 -0.752 -0.4444
## beta_1  0.5085  0.8516  1.0381  1.242  1.6365

```

Compare and interpret the estimates of β_0 and β_1 from the classical analysis and the Bayesian analysis.

```

# 'samples' is a mcmc.list so it needs to be a
# dataframe/matrix to pull values from
samples_matrix <- as.matrix(samples)

results <- data.frame(Parameter = c("beta_0", "beta_1"), GLM = coef(logistic_regression),
  JAGS = c(mean(samples_matrix[, "beta_0"]), mean(samples_matrix[,
    "beta_1"])), JAGS_Lower = c(quantile(samples_matrix[,
    "beta_0"], 0.025), quantile(samples_matrix[, "beta_1"],
    0.025)), JAGS_Upper = c(quantile(samples_matrix[, "beta_0"],
    0.975), quantile(samples_matrix[, "beta_1"], 0.975)))

# round our decimals for cleanliness' sake
results[, -1] <- round(results[, -1], 3)
results

```

```

##           Parameter      GLM   JAGS JAGS_Lower JAGS_Upper
## (Intercept)   beta_0 -0.902 -0.918    -1.418    -0.444
## x              beta_1  1.020  1.050     0.509     1.636

```

Our GLM does a good job of estimating β values. The Bayesian estimate from JAGS is less accurate but still captures the true values in its confidence intervals. Estimates for β_0 and β_1 differ by 0.006 and 0.021, respectively. Most importantly, both models correctly determined the sign of β_1 , showing increases in x are associated with an increasing probability of $y = 1$.

I was curious to see the JAGS output plotted. So here is a figure showing both regressions together. Gray region represents the uncertainty provided by the Bayesian model.

```

jags_predicted <- data.frame(x = seq(min(x), max(x), length.out = 100))

jags_predictions <- sapply(jags_predicted$x, function(x_value) {
  plogis(samples_matrix[, "beta_0"] + samples_matrix[, "beta_1"] *
    x_value)
})

jags_predicted$mean <- apply(jags_predictions, 2, mean)
jags_predicted$lower <- apply(jags_predictions, 2, quantile,
  0.025)
jags_predicted$upper <- apply(jags_predictions, 2, quantile,
  0.975)

ggplot() + geom_point(data = data, aes(x = x, y = y, color = "Observed data")) +
  geom_line(data = predicted_data, aes(x = x, y = y_hat, color = "GLM")) +
  geom_ribbon(data = jags_predicted, aes(x = x, ymin = lower,
    ymax = upper), alpha = 0.2) + geom_line(data = jags_predicted,
  aes(x = x, y = mean, color = "JAGS")) + labs(x = "x", y = "Probability of y = 1",
  color = "Model") + theme_minimal()

```

